

Ejercicio 1 · Arquitecturas y tecnologías de programación sobre clientes web

Trabajo de la unidad 1 de entorno cliente: el modelo cliente/servidor, los tres cambios hechos sobre el catálogo de ejemplo y las respuestas al cuestionario sobre los tiempos del navegador y los límites de esta arquitectura.

Cómo se reparte el trabajo entre cliente y servidor

En una aplicación web el trabajo se reparte entre el navegador del usuario (el entorno cliente) y el servidor remoto (el entorno servidor). En el cliente se monta la interfaz: la estructura con HTML5, la presentación con CSS3 y el comportamiento interactivo con JavaScript. En el servidor se ejecuta la parte que no conviene que vea el usuario, normalmente con lenguajes como PHP, Python, Java o Node.js, y desde ahí se accede a la base de datos.

La diferencia más visible entre los dos entornos es la visibilidad del código: el del cliente es público — cualquiera puede abrir las herramientas del desarrollador con F12 y leerlo o modificarlo—, mientras que el del servidor nunca llega al navegador. Por eso lo sensible (contraseñas, cobros, permisos) vive siempre en el servidor, y en el cliente se queda lo que tiene que ir rápido. El principio general lo resumiría así: lo que aporta inmediatez va en el cliente; lo que necesita seguridad y datos, en el servidor.

Los dos entornos corren el trabajo en sitios distintos, y eso también tiene consecuencias económicas. Ordenar o filtrar datos en el navegador se hace con la CPU y la memoria del dispositivo de quien visita la página, mientras que lo mismo en el servidor gastaría los recursos de la empresa. Cuanto más trabajo se lleve el cliente, más barato sale mantener la aplicación.

Los tres cambios sobre el catálogo

1 · Cambiar el orden de la lista

El catálogo se ordenaba de menos a más caro:

```
catalogoProductos.sort((a, b) => a.precio - b.precio);
```

Lo dejé al revés, de más caro a más barato, y ajusté el mensaje para que muestre primero el producto más caro:

```
catalogoProductos.sort((a, b) => b.precio - a.precio);
```

Esta ordenación la hace en el navegador del visitante, sin pedir nada al servidor.

2 · Filtrar por otra categoría

El filtro seleccionaba "Informática"; lo cambié a "Telefonía":

```
productosFiltrados = catalogoProductos.filter(prod => prod.categoria === "Telefonía");
```

El filtrado se hace entero en el cliente; el servidor no interviene.

3 · Subir el volumen de datos

Para probar los límites del navegador, multipliqué los registros generados:

```
const TOTAL_REGISTROS = 500000; // antes: 150000
```

Con este cambio se puede ver qué pasa con la memoria y el tiempo de respuesta al pasar de 150.000 a 500.000 productos.

Medición de tiempos en el navegador (Cuestión 1)

Enunciado. Anota el tiempo en ms que indica el navegador al procesar 150.000 elementos frente a 500.000. ¿El tiempo consumido crece de forma lineal o en una proporción distinta?

Para medirlo de verdad, usé IA para diseñar una pequeña herramienta que genera catálogos de 150.000, 200.000, 250.000 y 500.000 registros, los ordena, los filtra y va apuntando en una tabla los milisegundos de cada operación. Los botones van liberando la memoria de la pasada anterior para que cada prueba empiece limpia, y repetí varias pasadas por cada tamaño.

Al generar los registros, las cuatro pasadas por tamaño dieron estos tiempos:

Tamaño	Pasada 1	Pasada 2	Pasada 3	Pasada 4	Media	ms por cada 1.000 reg.
150.000	59,2	45,4	34,6	53,3	48,13	0,321
200.000	85,8	59,2	85,8	56,0	71,70	0,359
250.000	80,9	71,5	83,7	103,7	84,95	0,340
500.000	184,7	156,9	146,7	192,0	170,08	0,340

Y la ordenación y el filtrado (una pasada) tardaron:

Tamaño	Ordenación (ms)	Filtrado (ms)	Registros "Telefonía"
150.000	131,40	5,10	30.053 (20,0 %)
200.000	98,50	7,40	39.945 (20,0 %)
250.000	126,70	17,60	50.038 (20,0 %)
500.000	293,20	42,00	100.604 (20,1 %)

La generación crece de forma **lineal**: el coste por cada mil registros se mantiene casi clavado (entre 0,32 y 0,36 ms) y, al pasar de 150.000 a 500.000 —que son 3,33 veces más datos—, el tiempo subió 3,53 veces, muy cerca de lo que tocaría si fuera perfectamente proporcional:

Comparación	Ratio real	Ratio lineal esperado
200.000 / 150.000	1,49×	1,33×
250.000 / 150.000	1,77×	1,67×
500.000 / 150.000	3,53×	3,33×

La diferencia entre el ratio real y el esperado ronda el 6 %, lo normal cuando el ordenador comparte su tiempo con otras cosas. La ordenación es la que más tarda y la que más se nota al crecer el volumen: pasó de unos 131 ms con 150.000 registros a unos 293 ms con 500.000; ahí sí se le ve subir algo más que lineal. El filtrado, que recorre los registros de uno en uno, se comporta de forma parecida, aunque al medirlo una sola vez sus valores variaron más.

De propina, en los cuatro tamaños la categoría "Telefonía" se quedó siempre cerca del 20 % (30.053, 39.945, 50.038 y 100.604 registros), lo que cuadra con las cinco categorías y con que los productos se generen al azar.

Diez mil personas ordenando a la vez (Cuestión 2)

Enunciado. Si una tienda online tiene 10.000 usuarios consultando y reordenando el catálogo al mismo tiempo, ¿cuánto trabajo de cómputo ejecutan los navegadores de los 10.000 usuarios? ¿Qué beneficio supone para la empresa que este algoritmo se ejecute en el navegador de cada usuario en lugar de lanzar consultas ORDER BY continuas a la base de datos del servidor?

Los 10.000 usuarios ejecutan a la vez la misma ordenación y filtrado de la cuestión anterior, pero cada uno en su propio navegador y con la CPU y la memoria de su propio dispositivo. Son 10.000 trabajos iguales repartidos entre 10.000 equipos: si cada navegador tarda, por ejemplo, unos 131 ms en ordenar los 150.000 registros, el trabajo total es 10.000×131 ms, pero como cada usuario tiene el suyo, ninguno nota el esfuerzo de los demás.

Para la empresa el beneficio es que ese trabajo no lo hace el servidor:

- No tiene que ordenar ni filtrar el catálogo 10.000 veces: envía los datos una vez y cada navegador se ocupa de ellos.
- Como el trabajo se reparte entre las máquinas de los clientes, el servidor no se satura aunque entre mucha gente a la vez.
- Y queda libre para las tareas que solo él puede hacer con seguridad: cobros, accesos, contraseñas y datos importantes.

¿Y si hubiera ocho millones de productos? (Cuestión 4)

Enunciado. Si el inventario contara con 8 millones de registros, ¿sería viable descargarlos todos en un solo array en el navegador para que el cliente los ordene? ¿Qué solución propondrías?

No, no sería viable. Bajar ocho millones de registros de golpe al navegador tendría tres problemas: el ordenador o el móvil del usuario se quedaría sin memoria (miles y miles de objetos a la vez) y la página

podría quedarse colgada; la descarga duraría muchísimo y consumiría datos; y aunque llegara, ordenar y filtrar ocho millones de elementos tardaría demasiado como para que la página respondiera con fluidez.

La solución es no cargar todo el catálogo de una vez, sino irlo pidiendo por partes:

1. **Mostrar una parte cada vez.** Pedir al servidor los registros por grupos (por ejemplo, de 50 o 100 en 100) ya ordenados y filtrados, y pedir el siguiente grupo cuando haga falta. El navegador solo tiene un puñado en cada momento.
2. **Buscar en el servidor.** Para encontrar un producto concreto, preguntar al servidor y traer solo los resultados, no todos los datos para filtrarlos después en el cliente.
3. **Pintar lo que se ve.** No crear en la página los ocho millones de elementos, sino solo los que toca mostrar.

Con esto, el trabajo pesado (contar, ordenar y filtrar muchos registros) lo hace el servidor, que es quien está preparado para eso, y el navegador únicamente muestra la parte que se ve en cada momento. Se respeta la idea de que lo que necesita seguridad y volumen de datos va en el servidor.